

# Mastering GitHub Actions CI/CD: Workflows, Runners, Terraform & Secrets

A comprehensive guide to automating your development and infrastructure workflows using GitHub's powerful built-in CI/CD platform.

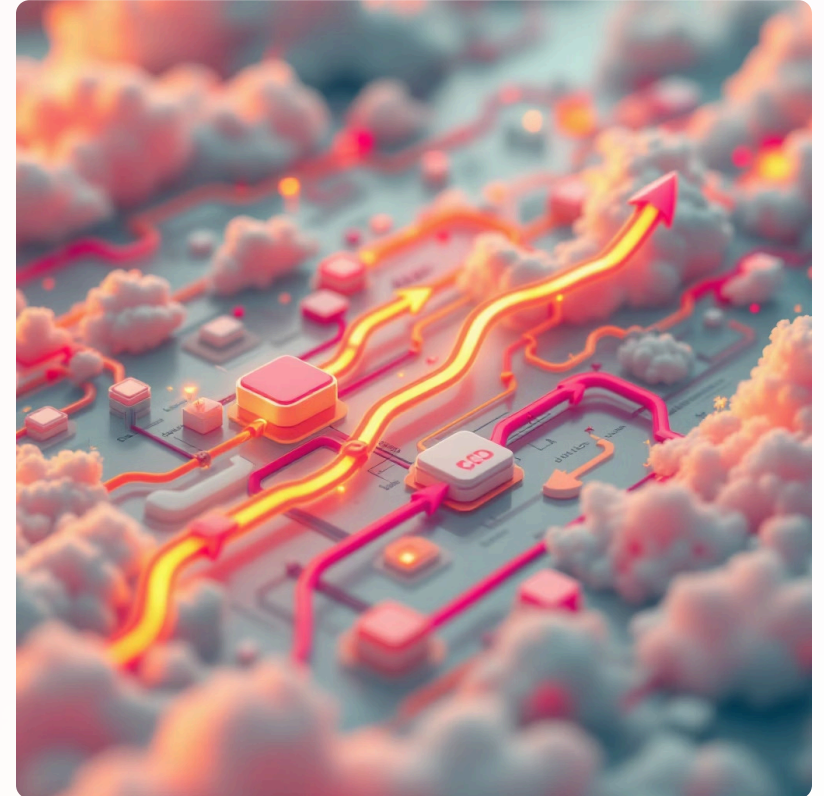


# What is GitHub Actions CI/CD?

GitHub Actions is a **native automation platform** built directly into GitHub repositories that enables:

- End-to-end CI/CD pipelines triggered by GitHub events (commits, PRs, schedules)
- Seamless DevOps workflows without external tool integration
- Automated building, testing, and deployment of applications

With GitHub Actions, your entire development lifecycle can be automated through code stored directly alongside your application.

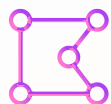


# Understanding GitHub Runners



## GitHub-Hosted Runners

Pre-configured virtual machines maintained by GitHub with Ubuntu Linux, Windows, and macOS options. Clean environment for each job with standard tooling installed.



## Self-Hosted Runners

Your own machines registered with GitHub. Ideal for custom hardware requirements, specific software needs, or running in private networks with specialized security.



## Runner Operation

Each runner executes a single job at a time, checking out your repository code, setting up specified environments, executing commands, and reporting results back to GitHub.

Runners provide the computing resources that power your CI/CD pipelines, executing the steps defined in your workflow files.

# Workflows & YAML Files: The Heart of Automation

GitHub Actions workflows are defined in YAML files stored in the `.github/workflows/` directory of your repository.

Key components include:

- **Triggers:** Events that start workflow runs (push, pull request, schedule)
- **Jobs:** Groups of steps that execute on the same runner
- **Steps:** Individual tasks that run commands or actions
- **Actions:** Reusable units of code for common tasks

```
name: Basic CI Workflow
```

```
on:
```

```
  push:
```

```
    branches: [ main ]
```

```
  pull_request:
```

```
    branches: [ main ]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Set up Node.js
```

```
        uses: actions/setup-node@v3
```

```
        with:
```

```
          node-version: '16'
```

```
      - run: npm ci
```

```
      - run: npm test
```



# Terraform Integration with GitHub Actions

## Infrastructure as Code Automation

GitHub Actions enables fully automated Terraform workflows for infrastructure provisioning, updating, and destruction while maintaining proper review processes.

## Typical Workflow Pattern

Generate and review Terraform plans on pull requests, then automatically apply verified changes when merged to the main branch.

## State Management

Store Terraform state remotely in Terraform Cloud, AWS S3, Azure Storage, or GCP Storage for collaboration, locking, and versioning.

# Sample Terraform Workflow Highlights



## Checkout Code

Retrieve repository code containing Terraform configurations



## Setup Terraform

Install and configure Terraform CLI with specified version



## Plan Changes

Initialize and run terraform plan to preview infrastructure changes



## Review

Post plan as PR comment for team review and approval

## Apply

Automatically apply approved changes when merged to main

```
# Terraform Plan Step
- name: Terraform Plan
  id: plan
  run: terraform plan -no-color
  continue-on-error: true

# Post Plan as PR Comment
- uses: actions/github-script@v6
  if: github.event_name == 'pull_request'
  with:
    script: |
      const output = `#### Terraform Plan
```

# Managing Secrets in GitHub Actions

Secrets provide a secure way to store sensitive data needed by your workflows:

- API keys, access tokens, and credentials
- Database connection strings
- Private keys for deployment
- Cloud provider authentication details

Secrets are **encrypted** at rest, masked in logs, and only exposed to specified jobs during runtime.

Access in workflows using:

```
${{ secrets.SECRET_NAME }}
```



# Repository & Environment Settings for Secure CI/CD

## Repository Secrets

Available to all workflows in a single repository. Ideal for project-specific credentials like API keys or deployment tokens.

**Example:** `DOCKERHUB_TOKEN` for pushing container images

## Environment Secrets

Tied to specific deployment environments with protection rules. Can require approvals before access.

**Example:** `PRODUCTION_DATABASE_URL` only available when deploying to production

## Organization Secrets

Centrally managed across an organization. Can be selectively shared with specific repositories.

**Example:** `AWS_ROLE_ARN` for standardized cloud access across teams

Environment protection rules can require specific reviewers to approve deployments, adding an extra layer of security for sensitive environments.



# Best Practices & Tips

## Security

- Use OIDC for cloud authentication instead of long-lived credentials
- Enable branch protection rules requiring PR reviews
- Set minimum approval count for sensitive environments
- Regularly rotate secrets and use short-lived tokens

## Workflow Efficiency

- Separate workflows by purpose (lint, test, deploy)
- Cache dependencies to speed up builds
- Use matrix builds to test across multiple configurations
- Set concurrency limits to prevent parallel deployments

# Conclusion: Automate Infrastructure with Confidence

**100%**

## **Code-Driven**

All infrastructure and deployment processes  
defined as versioned code

**0**

## **Manual Steps**

Eliminate error-prone manual processes  
with full automation

**24/7**

## **Availability**

Continuous monitoring and automated  
rollbacks ensure reliability

GitHub Actions CI/CD empowers teams to implement DevOps practices with infrastructure as code, security-first workflows, and environment management—all within the GitHub ecosystem where your code already lives.

Start building your automated infrastructure pipelines today to accelerate development cycles while maintaining security and compliance!